

Smart Device Firmware

Complete System Reference

Status	live on hardware as of 2026-08-29
Firmware version	1.0.0
Active kernel	custom microkernel mk_* v0.8.0 (Option A FreeRTOS port shim)
Active build	PlatformIO [env:esp32s3_arduino]
UART	Silicon Labs CP210x, COM5, 115200 baud
MAC (this board)	1c:db:d4:9c:1d:78
Board	ESP32-S3-DevKitC-1 v1.1, N8R8 (8MB Flash + 8MB Octal PSRAM)
Display	1.28" round TFT VER1.0, GC9A01, 240x240, 4-wire SPI

Canonical documentation — supersedes docs/hardware.md and README.md wiring for CS->GPIO9 / RST->GPIO14. Proven 5-wire map inside.

Table of Contents

1. Project Overview
 2. Hardware Identity
 3. Official DevKitC-1 v1.1 Header Map
 4. Working Display Wiring (Proven)
 5. Pin Mapping — Firmware vs Silk
 6. Why This Wiring Is Required
 7. System Architecture
 8. Dual-Kernel Design
 9. Custom Microkernel (mk_*)
 10. FreeRTOS Role
 11. Kernel Switch (NVS + Reboot)
 12. Two Firmware Tracks
 13. Boot Sequence
 14. Tasks, Priorities, Cores
 15. Display Pipeline
 16. Source Layout
 17. Serial Console
 18. Build, Flash, Monitor
 19. Official Documents
 20. Known Pitfalls
- Quick Card

1. Project Overview

Circular smart-display firmware targeting ESP32-S3-DevKitC-1 v1.1 with GC9A01 round display. Design goal: portable **mk_*** application API so product code never calls FreeRTOS directly, while Wi-Fi/BLE/USB/OTA stay on ESP-IDF FreeRTOS. Option A (shim) ships; Option B (from-scratch Xtensa scheduler) is roadmap.

Item	Value
MCU board	ESP32-S3-DevKitC-1 v1.1
Module	ESP32-S3-WROOM-1 N8R8 (silk MCN8R8)
CPU	Dual Xtensa LX7 @ 240 MHz
Flash	8 MB Quad SPI
PSRAM	8 MB Octal SPI (GPIO35/36/37 reserved)
Display	1.28" round TFT VER1.0, GC9A01, 240x240, 4-wire SPI
RGB LED	Addressable LED on GPIO38 (v1.1; v1.0 used GPIO48)
Product GUI (IDF)	LVGL 9.5 + glassmorphism dashboard
Bring-up GUI (Arduino, flashed)	Adafruit GC9A01A software SPI dashboard

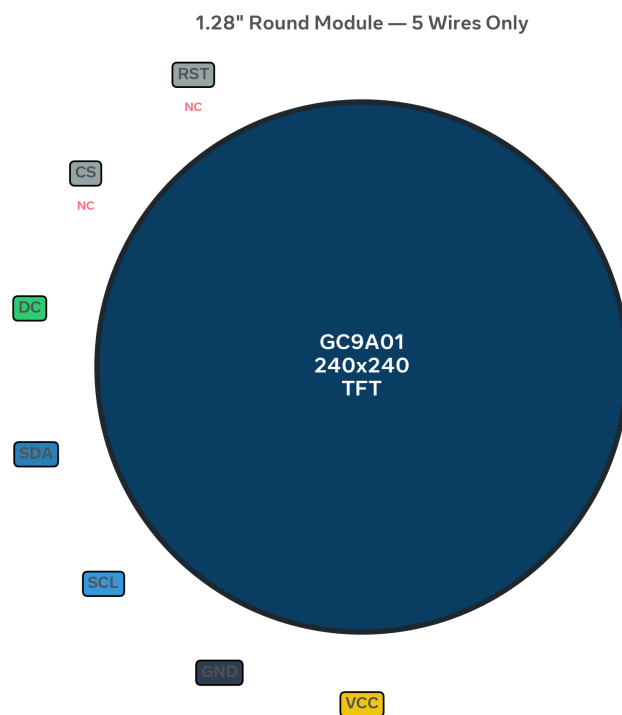
2. Hardware Identity

2.1 DevKit

ESP32-S3-DevKitC-1 v1.1 ordering code **ESP32-S3-DevKitC-1-N8R8**. Module WROOM-1-N8R8 with PCB antenna. Two Micro-USB: **UART** (CP210x up to 3 Mbps) and **USB** (native OTG GPIO19/20). 5V -> 3.3V LDO. Red power LED on 3.3V. RGB on GPIO38. Buttons BOOT (download) and RESET. 40MHz crystal. All GPIO 3.3V only — only 5V on board is VBUS broken out as silk **5V**.

2.2 Display Module

Back silk: **1.28" TFT VER1.0**, 240*240, IC: GC9A01, header marked SPI, pin order VCC toward RST: **VCC GND SCL SDA DC CS RST**. Note at R8: **R8 is CS pull-down; module may leave CS and RST unconnected**. No BLK pin — backlight LED tied to VCC. Feed 5V or glass stays dim while MCU runs. SCL/SDA on PCB are **SPI CLK and MOSI**, not I2C (extra DC/CS exist only on SPI panels).

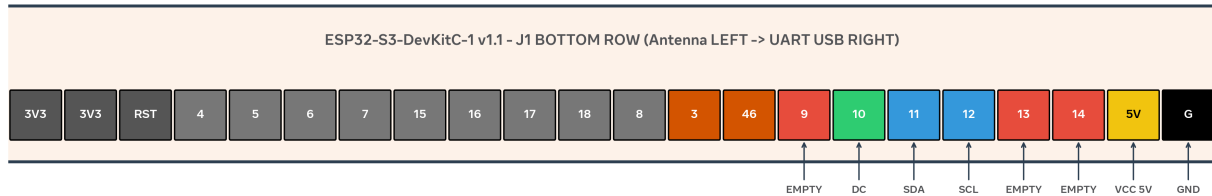


2.3 Pins That Must Never Be Used for LCD

GPIO	Why forbidden
35, 36, 37	Octal PSRAM inside WROOM-1-N8R8
19, 20	Native USB D- / D+
43, 44	UART0 TX/RX (console / flash)
0, 3, 45, 46	Strapping pins
38	RGB LED
21	GPIO21 on J3 right row — NOT 5V

3. Official DevKitC-1 v1.1 Header Map

Source: hardware screenshot/esp-dev-kits-en-master-esp32s3.pdf Chapter 1, Header Block, Fig.4 pin layout and https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/user_guide_v1.1.html. Hold board antenna/metal can at top, USB at bottom.



J1 No.	Silk	Function
1	3V3	3.3V antenna end
2	3V3	3.3V
3	RST	Chip EN (ESP reset, NOT LCD RST)
4	4	GPIO4
5	5	GPIO5
6	6	GPIO6
7	7	GPIO7
8	15	GPIO15
9	16	GPIO16
10	17	GPIO17
11	18	GPIO18
12	8	GPIO8
13	3	GPIO3 (strap)
14	46	GPIO46 (strap)
15	9	GPIO9 FSPIHD
16	10	GPIO10 FSPICS0
17	11	GPIO11 FSPID (MOSI)
18	12	GPIO12 FSPICLK
19	13	GPIO13 FSPIQ (MISO unused)
20	14	GPIO14 FSPIWP
21	5V	5V from USB VBUS
22	G	Ground

J3 = right row antenna-end G down to USB-end G: TX/RX, GPIO1-2, 42-35, 0,45,48,47, GPIO21, USB 20/19, G,G. Power options mutually exclusive: USB-to-UART and/or ESP USB; 5V+G; or 3V3+G.

GPIO	FSPI name	LCD use
11	FSPID	MOSI / display SDA

12	FSPICLK	CLK / display SCL
10	FSPICS0	DC (command/data)
13	FSPIQ	unused (write-only panel)
9	FSPIHD	leave open (CS)
14	FSPIWP	leave open (RST)

4. Working Display Wiring (Proven)

4.1 Desk Orientation

Metal can / antenna = **left**, two USB sockets = **right**, display sits **below** board, use bottom row J1. That row has 3V3 at antenna and 5V G at UART USB. Do NOT use top row TX RX 21 — silk 21 there is GPIO21.

4.2 Count from UART USB, Bottom Row

Count from UART	Silk	Display wire
1	G	GND
2	5V	VCC
3	14	empty
4	13	empty
5	12	SCL
6	11	SDA
7	10	DC
8	9	empty

4.3 Round PCB — Five Wires Only

Display silk	ESP silk	Notes
VCC	5V	Backlight on VCC. 3V3 looks dim blue disc.
GND	G	G next to 5V, USB corner
SCL	12	SPI clock
SDA	11	SPI MOSI. Not I2C.
DC	10	Command / data
CS	no wire	R8 holds CS low. Driving GPIO9 HIGH deselects panel.
RST	no wire	GPIO14 next to 5V/G; driving holds GC9A01 in reset (black glass)

4.4 Breadboard Rule

DevKit sits in breadboard. Each ESP pin common only with other holes in same 5-hole row. Plug Dupont in same row as counted pin. Do not use long red/blue power rails unless verified tied to 5V/G. One row off = firmware still runs (RGB/serial alive), glass stays flat blue. Unplug USB before moving 5V. Do not hot-swap VCC.

5. Pin Mapping — Firmware vs Silk

5.1 Arduino Track (Flashed, Working)

File: **src/main.cpp** Driver: **Adafruit_GC9A01A tft(CS, DC, MOSI, SCLK, RST, MISO)** — software SPI. Hardware-SPI constructor **Adafruit_GC9A01A(&SPI, dc, cs, rst)** re-inits SPI to ESP32-S3 default pins and blanks panel.

Signal	Define	Value
--------	--------	-------

MOSI	TFT_MOSI	11
SCLK	TFT_SCLK	12
DC	TFT_DC	10
CS	TFT_CS	-1 (not driven)
RST	TFT_RST	-1 (not driven)
GPIO9	pinMode(9, INPUT)	high-Z
GPIO14	pinMode(14, INPUT)	high-Z

5.2 IDF Product Track (Not Currently Flashed)

File: components/board/include/board_pins.h

Signal	Define	Value
MOSI	LCD_PIN_MOSI	11
SCLK	LCD_PIN_SCLK	12
DC	LCD_PIN_DC	10
CS	LCD_PIN_CS	9 (driver toggles if stays 9)
RST	LCD_PIN_RST	-1
BL	LCD_PIN_BL	-1
Pixel clock	LCD_PIXEL_CLOCK_HZ	2 MHz
I2C expansion	I2C_EXP_SDA/SCL	16 / 17 (not LCD)

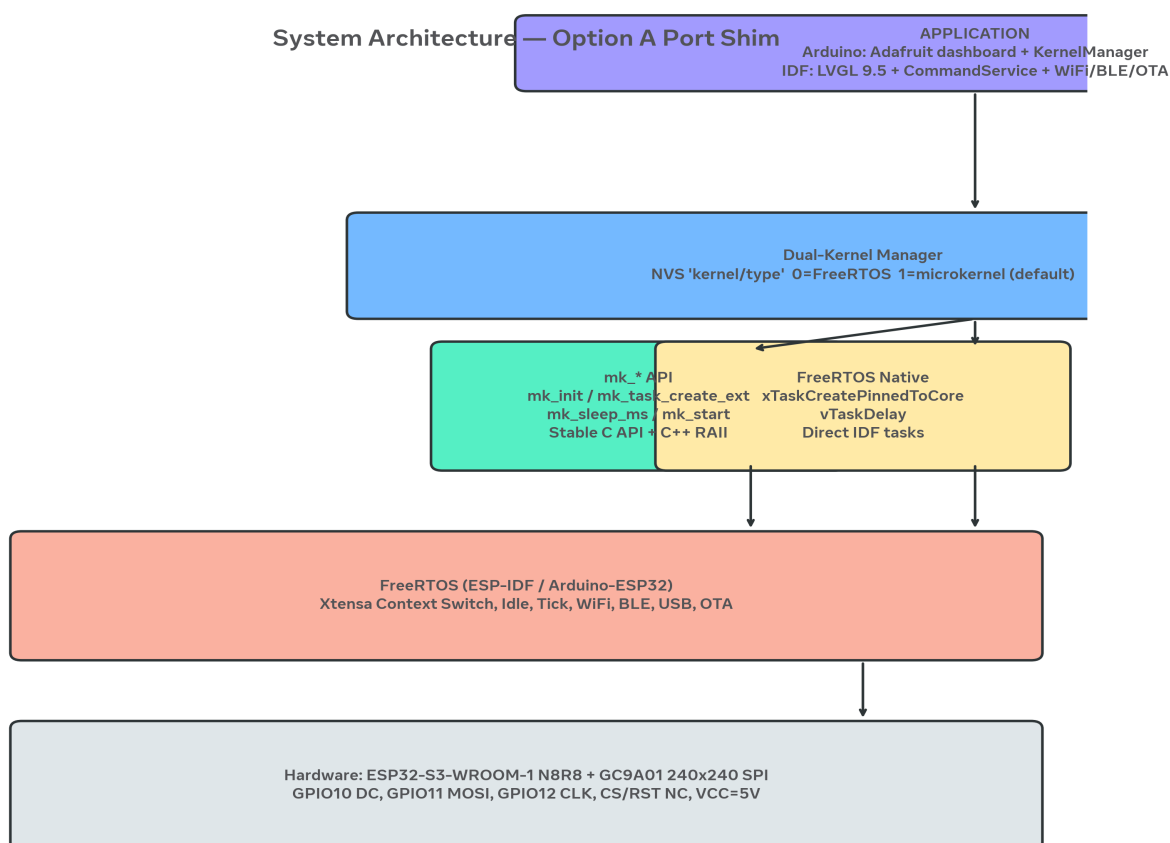
Hardware on desk must still leave CS and RST unplugged even if IDF driver drives GPIO9: R8 already selects chip. Do not jumper RST to 14.

6. Why This Wiring Is Required

Symptom	Cause	Fix
Dim / flat blue, MCU LED on	VCC on J1 3V3 antenna end	Move VCC to USB-end silk 5V
Completely black	LCD RST on GPIO14 (active-low glitches low ~60us)	Leave RST open; RST=-1
Backlight on, no pixels / snow	Duponts on antenna-end J1 while firmware bit-bangs 10/11/12	Move SCL/SDA/DC to silk 12/11/10 next to 5
Panel selected then dropped	Firmware driving GPIO9 as CS HIGH	TFT_CS=-1, CS header open
Adafruit HW SPI blank	Adafruit_GC9A01A(&SPI,...) remaps pins	Software-SPI constructor
GPIO21 used as 5V	J3 silk 21 is GPIO21	Only J1 silk 5V is 5V

GPIO9-14 are NOT strapping pins but DO have short low glitch at power-up (datasheet Table 2-2). Lethal if RST tied to GPIO14.

7. System Architecture



Layering for IDF product (main/app_main.cpp): APPLICATION -> SystemController -> GUI (LVGL 9.5 via lvgl_adapter) -> CommandService -> Services (WiFi, BLE, Time, OTA, Storage, Diagnostics) -> EventBus -> HAL (SPI, I2C, GPIO, USB) -> BoardConfig -> ESP-IDF + mk_* shim. Core split: Core0 ESP-IDF/Arduino system, WiFi, BLE, lwIP, SysMon, CLI; Core1 GUI (mk affinity 1).

8. Dual-Kernel Design

Both kernels are real runnable backends. Switching persisted and requires reboot because tasks already running cannot replace scheduler in place.

	Custom microkernel	Native FreeRTOS
NVS kernel/type	1 (default)	0
Task create	mk_task_create_ext	xTaskCreatePinnedToCore
Sleep	mk_sleep_ms	vTaskDelay
Dashboard chip	orange Microkernel	green FreeRTOS
Serial name	Custom Microkernel	FreeRTOS
Version string	0.8.0 (MK_CONFIG_VERSION_STRING)	tskKERNEL_VERSION_NUMBER
Context switch	FreeRTOS underneath shim	FreeRTOS directly

Files: src/kernel_manager.h/.cpp — Arduino dual-kernel; main/app_main.cpp — IDF dual-kernel; NVS namespace kernel, key type, uint8_t. Live confirmation after 2026-08-29 flash: Active: Custom Microkernel, Version 0.8.0, mk_task_count 3, CPU 2x Xtensa LX7 @240MHz.

9. Custom Microkernel (mk_*)

9.1 What It Is

Stable C API (mk_*) plus C++ RAI wrapper (mk_cpp.hpp) product code is supposed to call. Today every primitive is a port shim onto FreeRTOS (Option A). NOT a standalone Xtensa scheduler. No mk_context.S windowed-register save/restore yet. mk_start() sets running flag; does not take over CPU.

mk API	Shim implementation
mk_task_create_ext	xTaskCreate / xTaskCreatePinnedToCore
mk_sleep_ms	vTaskDelay
mk_yield	taskYIELD
mk_time_ms	esp_timer_get_time()/1000
mutex / sem / queue / event / timer	FreeRTOS equivalents (xSemaphoreCreateMutex has PI)
memory pool	custom pool on top of heap

9.2 Why Option A

ESP32-S3 WiFi, NimBLE, USB, OTA written against IDF FreeRTOS. Replacing scheduler is multi-year driver rewrite. Option A gives: app includes mk.h only (#error if freertos/*.h included unless MK_ALLOW_FREERTOS_SHIM), frozen mk_* v1 surface, later replacement of shim with real context switch without changing GUI/command/service code, WiFi/BLE keep working on Core0.

9.3 Tree

```
components/microkernel/
include/ mk.h, mk_config.h, mk_types.h, mk_kernel.h, mk_task.h, mk_mutex.h, mk_semaphore.h, mk_queue.h,
mk_event.h, mk_timer.h, mk_memory.h, mk_diagnostics.h, mk_cpp.hpp
core/ mk_kernel.c, mk_task.c, mk_scheduler.c, mk_idle.c
ipc/ mk_mutex.c, mk_semaphore.c, mk_queue.c, mk_event.c
time/ mk_tick.c, mk_timer.c
memory/ mk_pool.c
arch/esp32s3/ mk_arch.c, mk_port_shim.c
Arduino builds via tools/pio_kernel_sources.py (BuildLibrary + env.Prepend(LIBS=...)). IDF builds as
component microkernel.
```

9.4 Configuration (mk_config.h)

Macro	Value
Version	0.8.0
Tick	1000 Hz
Max tasks	16
Max priorities	8 (1=idle ... 7=highest)
Preemption	on
Round-robin	on
Mutex PI	on
Stack watermark	on

Watchdog timeout	10 s
------------------	------

Name	Value	Typical use
MK_PRIO_IDLE	1	idle
MK_PRIO_STORAGE	2	NVS / files
MK_PRIO_DIAGNOSTICS	3	SysMon / health
MK_PRIO_TIME	4	SNTP
MK_PRIO_CONNECTIVITY	5	WiFi supervisor
MK_PRIO_COMMAND	6	CLI
MK_PRIO_GUI	7	display / LVGL

Shim maps freertos_prio = 8 + mk_prio (GUI 7 -> FreeRTOS 15), capped at configMAX_PRIORITIES-1.

Macro	Size
MK_STACK_GUI	12288
MK_STACK_COMMAND	8192
MK_STACK_CONNECTIVITY	8192
others	4096

Arduino bring-up currently uses 8192 / 4096 / 4096 for GUI / SysMon / CLI.

9.5 API Surface

Kernel: `mk_init(const mk_config_t*)`, `mk_start()`, `mk_yield()/mk_sleep_ms(ms)`, `mk_time_ms()/mk_time_us()`, `mk_is_initialized()/mk_is_running()`, `mk_kernel_get_stats()`. Tasks: `mk_task_create` / `mk_task_create_ext`, `mk_task_delete` / `suspend` / `resume`, `mk_task_self` / `get_name` / `get_state` / `get_info`, `mk_task_count`. Self-delete single `vTaskDelete`. IPC: `mutex`, `semaphore`, `queue`, `event group`. Time/memory/diagnostics: `software timers`, `monotonic clock`, `fixed block pools` `mk_mem_pool_*`, `mk_malloc/mk_free`, `stack watermark`, `CPU load`, `watchdog register/feed`.

9.6 Scheduler Object

`mk_scheduler.c` keeps per-priority ready lists and context-switch counter. Pick-next is NOT CPU dispatcher. FreeRTOS still preempts. Lists exist so future real port can use same TCB layout.

9.7 Roadmap

Version	Intent
v0.8.0 (now)	Shim + PI mutex + stats + dual-kernel switch
v0.9	72h stress, fault injection, deadlock detect
v1.0	Freeze API, replace shim with Xtensa <code>mk_context.S</code> (window spill, PS/SAR, interrupt vs task stack)

Until v1.0, saying 'device runs custom microkernel' means app tasks are created and delayed through `mk_*`, not that FreeRTOS has been removed.

10. FreeRTOS Role

On ESP32-S3, FreeRTOS is OS inside Arduino-ESP32 and ESP-IDF. It owns Xtensa context switch and windowed ABI, SysTick/idle/timer daemon, task watchdog, WiFi, NimBLE, lwIP, event loop, Arduino loop() task. When user selects FreeRTOS via switch_kernel freertos: KernelManager does NOT call mk_init, GUI/SysMon/CLI are xTaskCreatePinnedToCore at prios 5/2/3, sleep vTaskDelay, dashboard chip green FreeRTOS. When microkernel (default): same three jobs created with mk_task_create_ext, each mk task still FreeRTOS task underneath, kernel_info reports both mk_task_count and FreeRTOS task table. Intentional coexistence, not bug.

11. Kernel Switch (NVS + Reboot)

```
device> switch_kernel microkernel
device> switch_kernel freertos
```

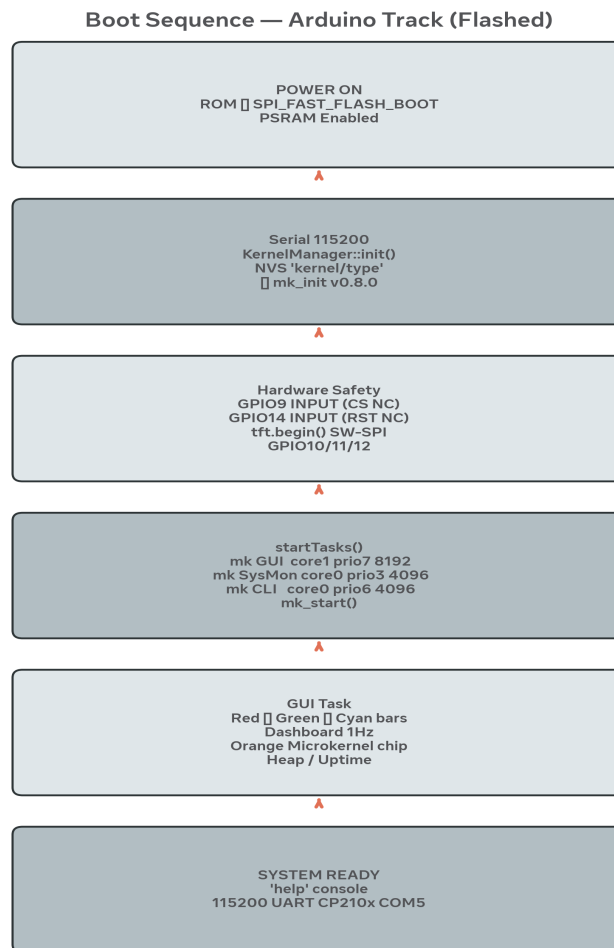
Implementation (KernelManager::switchKernel): 1) Compare with activeKernel; no-op if already selected 2) nvs_set_u8('kernel','type',0|1)+commit 3) ESP.restart(). Boot (KernelManager::init): 1) nvs_flash_init() 2) Read type; missing key -> microkernel 3) Persist choice sticky 4) If mk: mk_init then later startTasks -> mk_task_create_ext x3 -> mk_start. IDF app_main.cpp uses same NVS key (1=mk, default true if missing).

12. Two Firmware Tracks

	Arduino bring-up (on chip)	IDF product
Env	esp32s3_arduino (PlatformIO default)	esp32s3_idf / idf.py
Entry	main/arduino_main.cpp includes src/*	main/app_main.cpp
Display	Adafruit GC9A01A SW SPI	esp_lcd + esp_lcd_gc9a01 + LVGL 9.5
GUI	GFX dashboard 1Hz refresh	lvgl_adapter flush -> esp_lcd_panel_draw_bitmap
Commands	small Serial parser	CommandService (UART/USB CDC/BLE/WiFi)
WiFi/BLE/OTA	not in Arduino image	full services
Kernel	dual, mk default	dual, mk default

Arduino is path that compiled, flashed, painted round TFT on this PC. IDF image is intended product (LVGL, OTA, connectivity) once toolchain used.

13. Boot Sequence



13.1 Arduino (current flash)

POWER ON ROM->SPI_FAST_FLASH_BOOT PSRAM enabled -> setup() Serial 115200
KernelManager::init()->mk_init v0.8.0 display mutex GPIO9/14 INPUT tft.begin() SW SPI rotation 0 invert on
startTasks() mk GUI core1 mk SysMon core0 mk CLI core0 mk_start() 'SYSTEM READY' GUI:
red->green->cyan->black dashboard loop(): vTaskDelay(1000) (work in mk tasks)

13.2 IDF product

app_main load NVS kernel type mk_init platform, event loop, netif EventBus, NVS/settings (non-fatal) Board+SPI
HAL+I2C HAL+USB console GC9A01 init LVGL adapter Time/Diagnostics/OTA (non-fatal) spawn
GUI/SYSTEM/COMMAND(+WIFI) mk_start. NVS/time/diagnostics/OTA failures must not abort before first frame
(previously left panel in post-reset scan).

14. Tasks, Priorities, Cores

Arduino, kernel=microkernel (running)

mk name	Entry	mk prio	Core	Stack
---------	-------	---------	------	-------

GUI	gui_task	7	1	8192
SYSTEM	sys_monitor_task	3	0	4096
CLI	cli_task	6	0	4096

GUI: color bars once, then dashboard every 1s (kernel chip, heap, uptime). SysMon: g_seconds_counter++, log every 5s. CLI: Serial line parser.

Arduino, kernel=FreeRTOS

Name	prio	Core	Stack
gui_task	5	1	8192
sys_mon_task	2	0	4096
cli_task	3	0	4096

IDF product (when built)

Name	mk prio	Core
GUI	7	1
SYSTEM	3	1
COMMAND	6	1
WIFI_CONN	5	0

GUI loop: lv_timer_handler then mk_sleep_ms with minimum 5ms delay so IDLE1 not starved (task WDT).

15. Display Pipeline

Arduino (working)

```
gui_task
Adafruit_GC9A01A software SPI
GPIO12 CLK, GPIO11 MOSI, GPIO10 DC
CS/RST not driven
fillScreen / drawCircle / print
invertDisplay(true) (this GC9A01 module)
SPI is bit-banged so pin numbers in constructor are pins that toggle. Hardware SPI on this Adafruit port re-binds to S3
defaults and misses 11/12.
```

IDF

```
SpiHal (SPI2/FSPI)
-> esp_lcd panel IO
-> esp_lcd_gc9a01
-> LvglRuntime.flush_cb
-> esp_lcd_panel_draw_bitmap
Panel write-only; no readable chip ID. Success means ESP-side transactions completed.
```

16. Source Layout

```

smart_device_firmware/
platformio.ini Arduino default + IDF env
CMakeLists.txt IDF project
VERSION 1.0.0
src/ main.cpp Arduino dashboard + CLI, kernel_manager.cpp/.h dual-kernel + NVS switch
main/ arduino_main.cpp includes src/* for PIO src_dir=main, app_main.cpp IDF product
components/
microkernel/ mk_* API + shim
board/ pins + BoardConfig
device_hal/ SPI/I2C/GPIO/USB
display/ GC9A01 IDF driver
lvgl_adapter/ LVGL 9.5 no esp_lvgl_port
gui/ dashboard/status/commands
command/ unified CommandService
connectivity/ WiFi+NimBLE
storage/ NVS+settings
time_service/ SNTP
diagnostics/ health/self-test
ota/ dual-slot OTA
event_bus/ typed events
platform/ ESP32-S3 platform shim
common/ version Result<>
tools/ pio_kernel_sources.py compile mk into Arduino, flash.ps1/build.ps1/monitor.ps1
docs/ this file and topic docs
hardware screenshot/ photos + Espressif PDFs
simulator/ desktop UI/CLI

```

17. Serial Console

115200 8N1 on UART Micro-USB (CP210x).

Arduino image (what is on device)

Command	Action
help	list commands
status	kernel, heap, task count, uptime
tasks	FreeRTOS task count + handles
kernel_info	full kernel block
switch_kernel microkernel	persist type=1, reboot
switch_kernel freertos	persist type=0, reboot
test	cyan/green color bars
version	FW 1.0.0 + pin summary
reboot	ESP.restart()

IDF product (when flashed)

help, version, status, wifi scan/connect/status, ble start/stop/status, time status/sync, display test, display brightness 0-100, self-test, kernel status/tasks/stats, ota status/update, reboot, factory_reset. All IDF transports (UART, USB CDC, BLE, WiFi) share CommandService.

18. Build, Flash, Monitor

Arduino (this PC)

```
pio run -e esp32s3_arduino
pio run -e esp32s3_arduino -t upload --upload-port COM5
pio device monitor -p COM5 -b 115200
```

Do not toggle DTR/RTS into download after flash (boot:0x0 DOWNLOAD). Open monitor, or reset with GPIO0 high. Expected boot lines: SMART DEVICE — custom microkernel active, FreeRTOS selectable [KERNEL] Active: custom microkernel (mk_* API, FreeRTOS port shim) [KERNEL] mk_init v0.8.0 [TASK] GUI on core1 kernel=Custom Microkernel [KERNEL] mk_start — 3 mk tasks SYSTEM READY. Type help. Glass: red->green->cyan then dashboard with orange Microkernel chip.

IDF

```
idf.py set-target esp32s3
powershell -ExecutionPolicy Bypass -File tools/build.ps1
powershell -ExecutionPolicy Bypass -File tools/flash.ps1 -Monitor
```

19. Official Documents

File	What it is
hardware screenshot/esp-dev-kits-en-master-esp32s3.pdf	Board guide Ch.1=DevKitC-1 v1.1 Header Block J1/J3 RGB=GPIO38 5V=J1
hardware screenshot/esp32-s3_datasheet_en.pdf	Chip datasheet v2.2 GPIO 3.3V FSPI IOMUX no 5V GPIO octal PSRAM pow
hardware screenshot/ESP32-S3_DevKitC-1_pinlayout_v1.1.jpg	Official pin-layout art (USB at bottom)

Web: DevKitC-1 v1.1 user guide
https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/user_guide_v1.1.html
Schematic https://dl.espressif.com/dl/schematics/SCH_ESP32-S3-DevKitC-1_V1.1_20221130.pdf ESP32-S3
datasheet https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf Repo topic docs:
docs/architecture.md, docs/architecture_microkernel.md, docs/hardware.md (CS/RST rows outdated),
docs/commands.md, docs/ota.md, docs/device_mapping_and_state.md

20. Known Pitfalls

1. Antenna-end Duponts. Firmware talks to silk 10/11/12/5V/G. Wires next to metal can are 3V3/EN/GPIO4.
2. RST on 14. Black screen. Leave open.
3. CS driven high. Deselects panel whose R8 already holds CS low. Leave CS open on this module.
4. VCC on 3V3. Backlight too weak.
5. J3 silk 21 != 5V.
6. Adafruit hardware SPI on ESP32-S3.
7. Octal PSRAM GPIO35-37 must stay unused.
8. Download-mode serial open. RTS/DTR can leave boot:0x0 DOWNLOAD(USB/UART0). Reset with IO0 high.
9. Calling this a standalone kernel. It is mk_* on FreeRTOS shim until mk_context.S exists.
10. IDF GUI prio 15 with 1ms LVGL delay starves IDLE1 / task WDT. Floor delay at 5ms.

Quick Card — Print This

ESP32-S3-DevKitC-1 v1.1 WROOM-1 N8R8
Display 1.28" GC9A01 SPI (SCL/SDA=CLK/MOSI)

USB-end, bottom row, from UART:

G -> GND

5V -> VCC

14 empty 13 empty

12 -> SCL

11 -> SDA

10 -> DC

CS unplugged RST unplugged

Kernel default: custom mk_* v0.8.0 (FreeRTOS shim)

Switch: switch_kernel freertos | switch_kernel microkernel

Flash: pio run -e esp32s3_arduino -t upload --upload-port COM5

Console: 115200 8N1 CP210x COM5

Commands: help, status, kernel_info, version, test, reboot